



Pattern Recognition
and Applications Lab

La Programmazione ad Oggetti in Python

Docente: Ambra Demontis

Anno Accademico: 2025 - 2026

Corso di Laurea in Ingegneria Elettronica, Informatica e delle
Telecomunicazioni



University of Cagliari,
Italy

Department of Electrical and
Electronic Engineering



La Programmazione ad Oggetti in Python

In queste slide vedremo come:

- Sollevare eccezioni
- Gestire eccezioni
- Definire nuove eccezioni

Le Eccezioni

Durante l'esecuzione di un programma può capitare che alcuni input dell'utente o risultati di calcoli siano **invalidi** o **imprevisti**.

Ad esempio, dei risultati ottenuti possono portare alla divisione di un numero per zero.

Questi eventi generano spesso delle eccezioni.

Le Eccezioni

Alcuni esempi di eccezioni sono:

```
x = 5 / 0
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

x = 5 / 0

ZeroDivisionError: division by zero

```
lst = ["a", "b", "c"]
```

```
print(lst[3])
```

Traceback (most recent call last):

File "<stdin>", line 2, in <module>

print(lst[3])

IndexError: list index out of range

Le Eccezioni

Alcuni esempi di eccezioni sono:

```
a = [3] + 3
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

```
a = [3] + 3
```

TypeError: can only concatenate list (not "int") to list

Le Eccezioni

Le eccezioni:

1. Comunicano all'utente quale riga di codice ha generato il problema.
2. Comunicano qual'è il problema.
3. Terminano l'esecuzione del programma.

Esempio:

$x = 5 / 0$

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    x = 5 / 0
```

1

```
ZeroDivisionError: division by zero
```

2

Sollevare Eccezioni

Le eccezioni **possono essere sollevate dal programmatore per informare l'utente che è avvenuto un evento imprevisto.**

La sintassi per sollevare un'eccezione è:

`raise <tipo_eccezione> (“messaggio per l'utente”)`

Sollevare Eccezioni

In Python esistono diversi tipi di eccezioni. Quelli più comunemente utilizzati dai programmatori sono:

ValueError: un'operazione o funzione riceve un argomento che è di un tipo previsto ma ha un valore inappropriato.

TypeError: un'operazione o funzione è applicata ad un oggetto di tipo inappropriato.

NotImplementedError: indica che il valore ricevuto da un'operazione o da una funzione non è contemplato dall'attuale implementazione.

Sollevare Eccezioni

Consideriamo la classe CEsame implementata come mostrato sotto:

```
class CEsame:
    def __init__(self, nome, voto):
        self._nome = nome
        self._voto = voto

    @property
    def nome(self):
        return self._nome

    @property
    def voto(self):
        return self._voto
```

Sollevare Eccezioni

Supponiamo di voler far sì che venga generata un'eccezione se:

- 1) Il voto non è compreso tra 0 e 30 (il nostro programma prevede voti in questo range)
- 2) Il voto non è un intero

Controllo di Tipo

Per controllare se un oggetto appartiene esattamente al tipo desiderato possiamo utilizzare la funzione **type** e l'operatore **is** in questo modo:

```
type(<oggetto>) is <tipo_desiderato>
```

Esempio:

```
if type(3) is int:  
    print("tipo intero")  
else:  
    print("altro tipo")
```

Stamperà: tipo intero

Controllo di Tipo

```
if type(3.5) is float:  
    print("tipo frazionario")  
else:  
    print("altro tipo")  
Stamperà: tipo frazionario
```

```
if type("LPO") is str:  
    print("tipo stringa")  
else:  
    print("altro tipo")  
Stamperà: tipo stringa
```

Sollevare Eccezioni

Il valore dell'attributo che vogliamo controllare viene memorizzato nel metodo `__init__`

```
class CEsame:
```

```
    def __init__(self, nome, voto):  
        self._nome = nome  
        self._voto = voto
```

```
    ...
```

Quindi è sufficiente modificare questo metodo.

Sollevare Eccezioni

Possiamo far sì che questo metodo richiami una funzione che si occupa di effettuare i controlli sul valore di voto ed eventualmente sollevare un'eccezione.

```
class CEsame:
```

```
    def __init__(self, nome, voto):
```

```
        self._controlla_valore_voto(voto)
```

```
        self._nome = nome
```

```
        self._voto = voto
```

```
        ...
```

Sollevare Eccezioni

```
def _controlla_valore_voto(self, voto):  
  
    if type(voto) is not int:  
        raise TypeError("Il valore dell'attributo voto deve essere un "  
                        "intero")  
  
    if voto < 0 or voto > 30:  
        raise ValueError("Il valore dell'attributo voto deve essere "  
                        "compreso tra zero e 30")
```

Sollevare Eccezioni

Avendo modificato il codice come mostrato verrà sollevata un'eccezione nel caso in cui il valore dell'attributo voto passato come argomento al metodo `__init__` non sia conforme a quanto il programmatore si aspetta.

Esempio:

```
oggetto_esame = CEsame('LPO', "50")
```

Traceback (most recent call last):

...

TypeError: Il valore dell'attributo voto deve essere un intero

Perchè Sollevare Eccezioni?

Non potremmo semplicemente inserire una stampa di errore?

```
def _controlla_valore_voto(self, voto):  
  
    if type(voto) is not int:  
        print("Il valore dell'attributo voto deve essere un intero")  
  
    if voto < 0 or voto > 30:  
        print("Il valore dell'attributo voto deve essere compreso tra zero e 30")
```

NO, non si può fare!

- 1) Il programma continuerebbe generando risultati errati.
- 2) La stampa di errore potrebbe non essere vista dall'utente che utilizzerebbe i risultati errati generati dal programma.

Perchè Sollevare Eccezioni?

- Permette di **terminare subito il programma, liberando le risorse e segnalando** all'utente **che è avvenuto un evento imprevisto** evitando che questo continui con valori invalidi generando dei risultati errati*.
- Permette di fornire all'utente dei messaggi di errore ad-hoc e più informativi di quelli che potrebbero venire generati altrimenti.

* Sempre più spesso i calcoli vengono effettuati su costose GPU. Tenerle occupate inutilmente con un programma che non andrà a buon fine costituisce uno spreco di risorse.

Esercizio: Sollevare Eccezioni

Creare una classe chiamata User con un attributo modificabile chiamato username.

L'attributo deve essere una stringa e la sua lunghezza deve essere compresa tra 3 e 5 caratteri.

Altrimenti sollevare le apposite eccezioni.

Soluzioni: Sollevare Eccezioni

```
class CUtente:

    def __init__(self, username):
        self.username = username

    @property
    def username(self):
        return self._username

    @username.setter
    def username(self, username):

        if type(username) is not str :
            raise TypeError("L'username deve essere una stringa")

        if len(username) < 3 or len(username) > 5:
            raise ValueError("L'username deve essere composto da almeno 3 e massimo 15 caratteri")

        self._username = username
```

Gestire Eccezioni

Le eccezioni possono essere gestite dal programmatore.

Scrivendo del codice ad-hoc può intercettarle, evitando che il programma si blocchi ed effettuare delle operazioni per risolvere i problemi da esse segnalati.

Gestire Eccezioni

Nel caso in cui si voglia gestire con lo stesso codice tutte le eccezioni causate da ogni evento imprevisto generato dal codice si utilizza la sintassi:

try:

... (codice da eseguire se l'eccezione non si verifica)

except Exception:

... (codice da eseguire se un'eccezione di un tipo qualsiasi si verifica)

Gestire Eccezioni

Si può però anche decidere di gestire solo l'eccezione di un determinato tipo.
In questo caso si utilizza la sintassi:

try:

... (codice da eseguire se l'eccezione non si verifica)

except <tipo_eccezione>:

... (codice da eseguire se un'eccezione di tipo <tipo_eccezione> si verifica)

Gestire Eccezioni

Oppure di gestire l'eccezione di una lista di tipi di eccezioni.

In questo caso si utilizza la sintassi:

try:

... (codice da eseguire se l'eccezione non si verifica)

except (<tipo_eccezione_1>, .. <tipo_eccezione_n>):

... (codice da eseguire se un'eccezione di uno dei tipi specificati si verifica)

Gestire Eccezioni

Nel caso in cui si vogliano gestire tipi di eccezioni differenti svolgendo nel caso in cui essi si verifichino operazioni differenti:

try:

... (codice da eseguire se l'eccezione non si verifica)

except (<tipo_eccezione_1>, .. <tipo_eccezione_n>):

... (codice da eseguire se un'eccezione di uno dei tipi specificati si verifica)

except (<tipo_eccezione_n+1>, .. <tipo_eccezione_z>):

... (codice da eseguire se un'eccezione di uno dei tipi specificati si verifica)

Gestire Eccezioni

Ad esempio, supponiamo il gestore di un negozio abbia la necessità di avere una classe *StatisticheAcquisti* che gli permetta di ricevere, in fase di inizializzazione una lista (*lista_prezzi*) contenente i prezzi dei prodotti acquistati da un suo cliente e calcolare delle statistiche.

In particolare, questa classe deve avere due metodi pubblici:

- calcola_prezzo_medio* che permette di calcolare il prezzo medio della lista memorizzata.
- stampa_prezzo_medio* che permette di stampare a schermo il prezzo medio della lista memorizzata.

Gestire Eccezioni

CStatisticheAcquisti
- lista_prezzi
+ calcola_prezzo_medio() + stampa_prezzo_medio()

Supponiamo anche di essere certi che quando viene invocato il metodo `stampa_prezzo_medio`, qualcuno sia davanti al terminale.

Gestire Eccezioni

```
class CStatisticheAcquisti():
```

```
    def __init__(self, lista_prezzi):  
        self._lista_prezzi = lista_prezzi
```

```
    def calcola_prezzo_medio(self):  
        prezzo_totale = 0  
        for prezzo in self._lista_prezzi:  
            prezzo_totale = prezzo_totale + prezzo  
        return prezzo_totale / len(self._lista_prezzi)
```

...

Gestire Eccezioni

```
def stampa_prezzo_medio(self):  
    prezzo_medio = self.calcola_prezzo_medio()  
    print("Il prezzo medio è ", prezzo_medio)
```

Esempio:

```
oggetto_stat_acquisti = CStatisticheAcquisti([4, 8])  
oggetto_stat_acquisti.stampa_prezzo_medio()
```

Stampa:

Il prezzo medio è 6.0

Gestire Eccezioni

Questo codice, se riceve una lista invalida genera un'eccezione.

```
oggetto_stat_acquisti = CStatisticheAcquisti(['2'])
```

```
oggetto_stat_acquisti.stampa_prezzo_medio()
```

...

TypeError: unsupported operand type(s) for +: 'int' and 'str'

```
oggetto_stat_acquisti = CStatisticheAcquisti([])
```

```
oggetto_stat_acquisti.stampa_prezzo_medio()
```

...

ZeroDivisionError: division by zero

Gestire Eccezioni

Supponiamo si voglia far sì che, se viene inserita per sbaglio una lista invalida, ad esempio una lista vuota o una lista che contiene dei caratteri, il programma che richiama la funzione `stampa_prezzo_medio` non si blocchi ma stampi semplicemente a schermo il messaggio:

Non posso calcolare il prezzo medio per questa lista.

Gestire Eccezioni

Modifichiamo il metodo *stampa_prezzo_medio*:

```
def stampa_prezzo_medio(self):
```

```
    try:
```

```
        prezzo_medio = self.calcola_prezzo_medio()
```

```
        print("Il prezzo medio è ", prezzo_medio)
```

```
    except Exception:
```

```
        print("Non posso calcolare il prezzo medio per questa lista.")
```


Gestire Eccezioni

Dopo questa modifica, il seguente codice:

```
oggetto_stat_acquisti = CStatisticheAcquisti([4, 8])  
oggetto_stat_acquisti.stampa_prezzo_medio()
```

```
oggetto_stat_acquisti = CStatisticheAcquisti([])  
oggetto_stat_acquisti.stampa_prezzo_medio()
```

```
oggetto_stat_acquisti = CStatisticheAcquisti(['a'])  
oggetto_stat_acquisti.stampa_prezzo_medio()
```

Gestire Eccezioni

Stamperà:

Il prezzo medio è 6.0

Non posso calcolare il prezzo medio per questa lista.

Non posso calcolare il prezzo medio per questa lista.

Gestire Eccezioni

Perchè modificare il metodo *stampa_prezzo_medio* e non modificare invece *calcola_prezzo_medio* così che qualsiasi codice che richiede il calcolo del prezzo medio e quindi richiama il metodo *calcola_prezzo_medio* non si blocchi?

NB: è una cattiva idea ma è importante capire per quale motivo lo è!

Gestire Eccezioni

```
class CStatisticheAcquisti():  
    def __init__(self, lista_prezzi):  
        self._lista_prezzi = lista_prezzi
```

```
    def calcola_prezzo_medio(self):  
        try:  
            prezzo_totale = 0  
            for prezzo in self._lista_prezzi:  
                prezzo_totale = prezzo_totale + prezzo  
            return prezzo_totale / len(self._lista_prezzi)
```

```
        except Exception:  
            print("Non posso calcolare il prezzo medio per questa lista.")
```

NB: è una cattiva idea ma è importante capire per quale motivo lo è!

Gestire Eccezioni

```
class CStatisticheAcquisti():
    def __init__(self, lista_prezzi):
        self._lista_prezzi = lista_prezzi

    def calcola_prezzo_medio(self):
        try:
            prezzo_totale = 0
            for prezzo in self._lista_prezzi:
                prezzo_totale = prezzo_totale + prezzo
            return prezzo_totale / len(self._lista_prezzi)
        except Exception:
            print("Non posso calcolare il prezzo medio per questa lista.")
```

NB: è una cattiva idea ma è importante capire per quale motivo lo è!

NB: nel caso in cui l'eccezione si verifichi, il metodo `calcola_prezzo_medio` non richiama la `return` e quindi restituisce `None`

Gestire Eccezioni

```
def stampa_prezzo_medio(self):  
    prezzo_medio = self.calcola_prezzo_medio()  
    if prezzo_medio is not None:  
        print("Il prezzo medio è ", prezzo_medio)
```

Anche il metodo `stampa_prezzo_medio` va quindi modificato per tenere in considerazione il fatto che l'altro metodo potrebbe generare un'eccezione.

NB: è una cattiva idea ma è importante capire per quale motivo lo è!

Gestire Eccezioni

Questa soluzione potrebbe essere praticabile se il metodo *calcola_prezzo_medio* fosse privato e quindi fossimo sicuri che tutti i metodi che utilizzano quel metodo (nel nostro caso *stampa_prezzo_medio*) sono in grado di comportarsi in modo corretto nel caso in cui l'eccezione avvenga e venga gestita dal metodo *calcola_prezzo_medio*.

Poichè *calcola_prezzo_medio* è un metodo pubblico può potenzialmente venire invocato da qualsiasi codice quindi non abbiamo questa certezza!

E' quindi molto meglio far sì che *calcola_prezzo_medio* generi l'eccezione e siano i metodi che lo utilizzano a gestirla.

Gestire Eccezioni

Anche se *calcola_prezzo_medio* fosse privato sarebbe comunque meglio siano i metodi che lo utilizzano e che certamente verranno invocati quando un utente è al terminale* a gestirla (è un metodo per il calcolo e potremmo dimenticarci di aver inserito lì la gestione dell'eccezione).

*ad esempio i metodi che effettuano stampe a schermo o che interagiscono con l'utente.

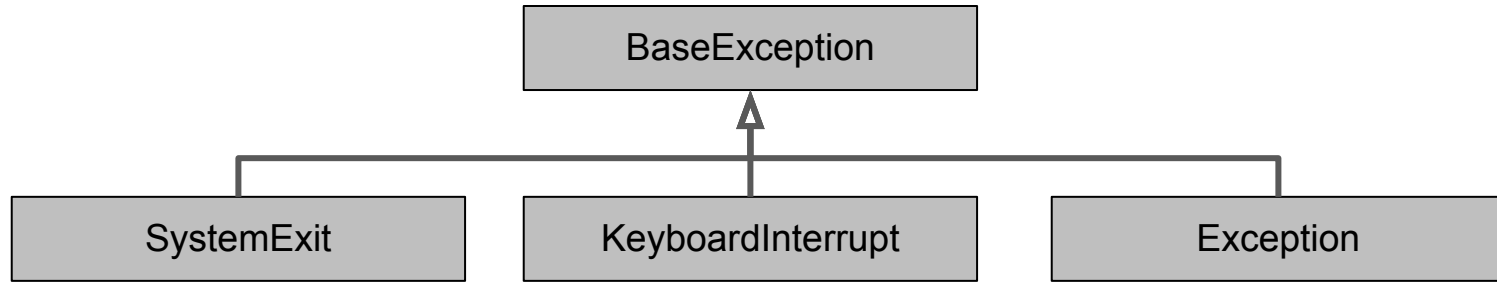
Definire Nuove Eccezioni

Le eccezioni sono oggetti.

Quando utilizziamo il codice per sollevare un'eccezione, utilizzando la sintassi: `raise <tipo_eccezione> (“messaggio per l'utente”)` stiamo in realtà creando un oggetto appartenente alla classe `<tipo_eccezione>` e passando al metodo iniziatore il messaggio per l'utente.

E' quindi possibile definire nuove eccezioni ereditando dalle classi esistenti per la gestione delle eccezioni.

Definire Nuove Eccezioni



- **SystemExit** sono le eccezioni che vengono sollevate quando il programma termina naturalmente (senza errori).
- **KeyboardInterrupt** sono le eccezioni che vengono sollevate quando il programma viene terminato dall'utente con una combinazione di tasti (es: CTRL+C).
- **Exception** sono tutte le altre eccezioni.

Definire Nuove Eccezioni

Per definire una nuova eccezione si crea una classe che eredita dalla classe Exception.

Ad esempio se volessimo creare un'eccezione per un programma di gestione di un bancomat chiamata PrelievoInvalido potremmo semplicemente definirla come:

```
class CPrelievoInvalido(Exception):  
    pass
```

Definire Nuove Eccezioni

Poichè eredita dalla classe Exception, quando viene sollevata, ad esempio dal codice:

```
raise CPrelievoInvalido("Il prelievo richiesto non può essere effettuato")
```

Come per le altre eccezioni, il programma termina e viene stampato a schermo il messaggio di errore.

Es:

...

```
__main__.CPrelievoInvalido: Il prelievo richiesto non può essere effettuato
```

Esercizio: Definire Nuove Eccezioni

Supponiamo di voler modificare la classe creata precedentemente in modo da far sì che stampi sempre a schermo il messaggio:

Il prelievo richiesto non può essere effettuato. Hai cercato di prelevare x euro più di quelli presenti sul tuo conto.

Nb: il testo del messaggio vogliamo sia sempre lo stesso, mentre deve cambiare il valore di x mostrato nel messaggio.

Soluzione: Definire Nuove Eccezioni

```
class CPrelievoInvalido(Exception):
```

```
    def __init__(self, importo_non_presente):
```

```
        messaggio = "Il prelievo richiesto non può essere effettuato. Hai cercato di prelevare " + \
                    str(importo_non_presente) + \
                    " euro più di quelli presenti sul tuo conto."
```

```
        super().__init__(messaggio)
```

Soluzione: Definire Nuove Eccezioni

```
raise CPrelievoInvalido(100, 150)
```

Stampa:

__main__.CPrelievoInvalido: Il prelievo richiesto non può essere effettuato. Hai cercato di prelevare 50 euro più di quelli presenti sul tuo conto.

Esercizio: Gestire Eccezioni

Gestire le eccezioni nel seguente codice facendo in modo che l'utente possa reinserire i valori

```
class UsernameError( Exception ):
    pass

class PasswordError( Exception ):
    pass
```


Esercizio: Gestire Eccezioni

```
class CLoginSystem:
    def __init__(self, username, password):
        self.username = username
        self.password = password

    def login(self, username, password):

        if username != self.username:
            raise UsernameError("Username non trovato.")

        if password != self.password:
            raise PasswordError("Password incorretta.")

        return True
```

Esercizio: Gestire Eccezioni

```
class CLoginManager:

    def __init__(self, login_system):
        self._login_system = login_system

    def login_da_tastiera(self):
        username = input("inserisci l'username ")
        password = input("inserisci la password ")
        self._login_system.login(username, password)
        return True

login_system = CLoginSystem("lala", "m14p4sw0rd")
login_manager = CLoginManager(login_system)
login_manager.login_da_tastiera()
```

Soluzione: Gestire Eccezioni

```
class CLoginManager:

    def __init__(self, login_system):
        self._login_system = login_system

    def login_da_tastiera(self):
        login = None
        while login is None:
            try:
                username = input("inserisci l'username ")
                password = input("inserisci la password ")
                login = self._login_system.login(username, password)

            except UsernameError:
                print("l'username è errato")
            except PasswordError:
                print("la password è errata")
        return True
```